

## Program 6:

Deploy an Automated Build Pipeline using Docker Hub

### Project Structure:

program\_6/

—>Dockerfile

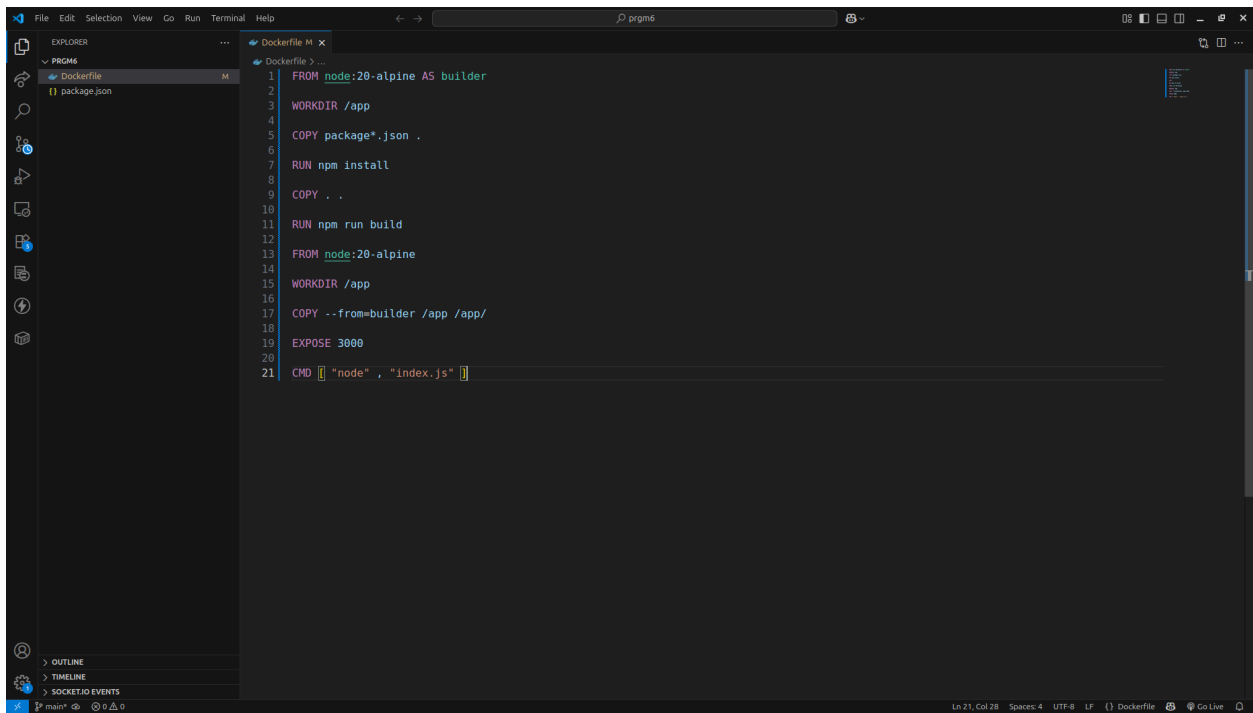
—>app.py

—>requirements.txt

### Procedure:

Create a folder (ex: program\_6)

Step 1: Create a simple Dockerfile without any extensions



The screenshot shows a Visual Studio Code editor window with a Dockerfile open. The Dockerfile contains the following instructions:

```
1 FROM node:20-alpine AS builder
2
3 WORKDIR /app
4
5 COPY package*.json .
6
7 RUN npm install
8
9 COPY . .
10
11 RUN npm run build
12
13 FROM node:20-alpine
14
15 WORKDIR /app
16
17 COPY --from=builder /app /app/
18
19 EXPOSE 3000
20
21 CMD ["node", "index.js"]
```

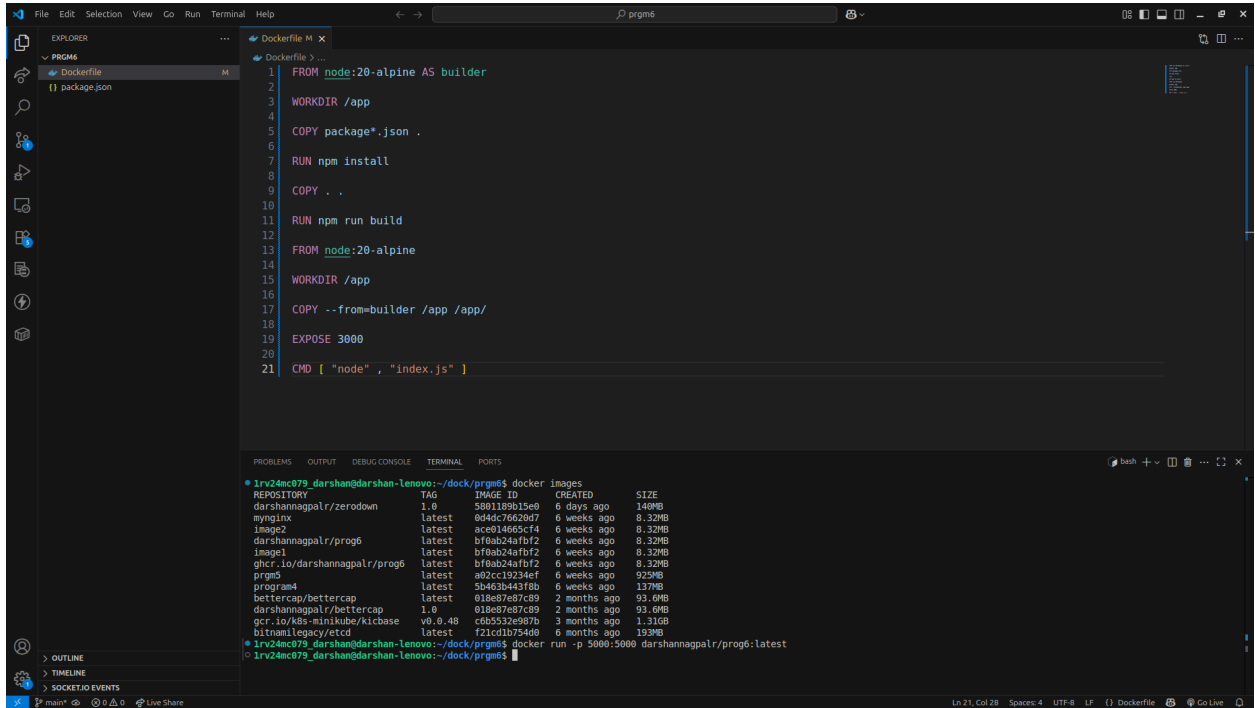
The Explorer sidebar on the left shows a folder named 'program\_6' containing 'Dockerfile' and 'package.json'. The bottom status bar indicates the current line is 21, column 28, with 4 spaces, in UTF-8 encoding, LF line endings, and the file is named 'Dockerfile'.

Build an image

`docker build -t program_6 .`

Run the container

`docker run -p 5000:5000 program_6`



The screenshot shows a VS Code editor with a Dockerfile open in the editor pane. The Dockerfile contains the following instructions:

```
1 FROM node:20-alpine AS builder
2
3 WORKDIR /app
4
5 COPY package*.json .
6
7 RUN npm install
8
9 COPY . .
10
11 RUN npm run build
12
13 FROM node:20-alpine
14
15 WORKDIR /app
16
17 COPY --from=builder /app /app/
18
19 EXPOSE 3000
20
21 CMD [ "node", "index.js" ]
```

The Explorer pane on the left shows a file named `package.json`. The Terminal pane at the bottom shows the output of the `docker images` command, listing various Docker images. The last line of the terminal output shows the command `docker run -p 5000:5000 darshannagpalr/program6:latest` being executed.

| REPOSITORY                      | TAG     | IMAGE ID     | CREATED      | SIZE   |
|---------------------------------|---------|--------------|--------------|--------|
| REPOSITORY                      | TAG     | IMAGE ID     | CREATED      | SIZE   |
| darshannagpalr/zerodown         | 1.0     | 5001189b15e0 | 6 days ago   | 140MB  |
| mynginx                         | latest  | 0d4dc76620d7 | 6 weeks ago  | 8.32MB |
| image2                          | latest  | ace014665c74 | 6 weeks ago  | 8.32MB |
| darshannagpalr/program6         | latest  | b10ab24afbf2 | 6 weeks ago  | 8.32MB |
| image1                          | latest  | b10ab24afbf2 | 6 weeks ago  | 8.32MB |
| ghcr.io/darshannagpalr/program6 | latest  | b10ab24afbf2 | 6 weeks ago  | 8.32MB |
| program5                        | latest  | a02cc19234ef | 6 weeks ago  | 925MB  |
| program4                        | latest  | 5b463b43f8b  | 6 weeks ago  | 137MB  |
| bettercap/bettercap             | latest  | 018e87e87c89 | 2 months ago | 93.6MB |
| darshannagpalr/bettercap        | 1.0     | 018e87e87c89 | 2 months ago | 93.6MB |
| gcr.io/k8s-minikube/kicbase     | v0.0.48 | c05532e9070  | 3 months ago | 1.31GB |
| bitnamilegacy/etcd              | latest  | f21cd1b75400 | 6 months ago | 193MB  |

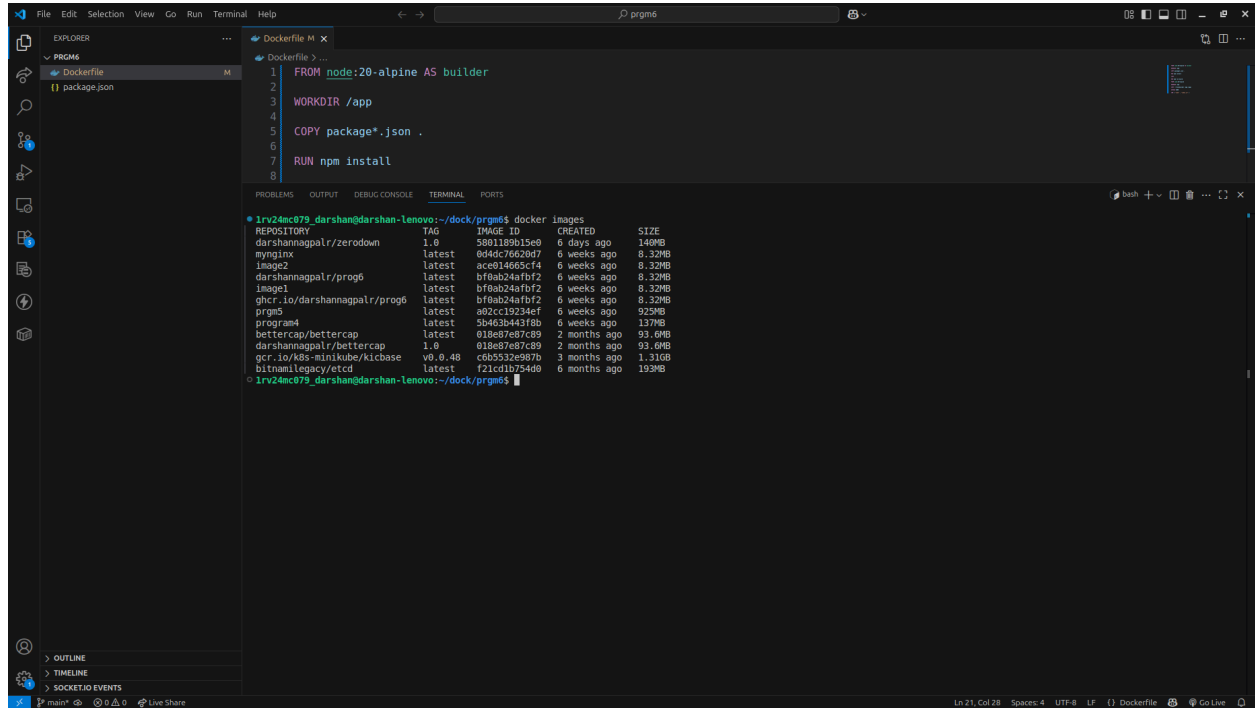
Terminal Output:

```
1rv24mc079_darshan@darshan-lenovo:~/dock/prgm6$ docker images
1rv24mc079_darshan@darshan-lenovo:~/dock/prgm6$ docker run -p 5000:5000 darshannagpalr/program6:latest
```

Step 2: Tag the created image for Docker Hub

`docker tag program_6 <dockerhub-username>/program_6:latest`

Step 3: Tag the Image for GitHub Container Registry (GHCR)  
docker tag program\_6 ghcr.io/<github-username>/program\_6:latest  
Push the Image to GHCR  
echo \$GITHUB\_PAT | docker login ghcr.io -u <github-username> - -



Step 4: Push source code to GitHub